

**Федеральное государственное автономное
образовательное учреждение высшего образования
«Санкт-Петербургский национальный исследовательский университет
информационных технологий, механики и оптики»**



Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №1

по дисциплине

«Функциональная схемотехника»:

Вариант 4

Выполнили:

Ястребов-Амирханов Алекси

Исупов Никита Александрович

Поток: 1.1

ПРЕПОДАВАТЕЛЬ: Табунщик Сергей Михайлович

Санкт-Петербург,

2025

Цель работы:

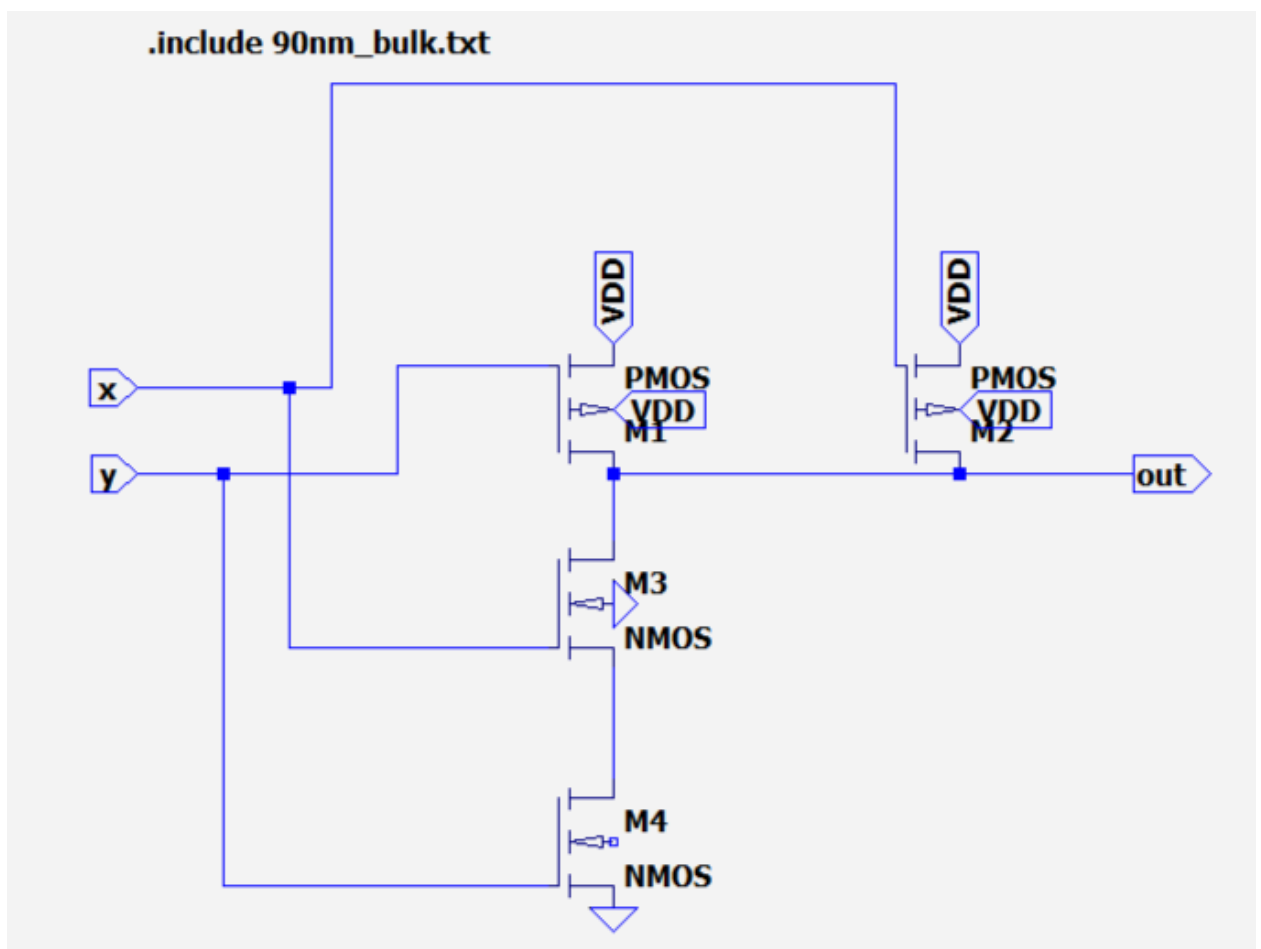
Получить базовые знания о принципах построения цифровых интегральных схем с использованием технологии КМОП. 2. Познакомиться с технологией SPICE-моделирования схем на транзисторах. 3. Получить навыки описания схем базовых операционных элементов (БОЭ) ком бинационного типа на вентиляном уровне с использованием языка описания аппаратуры Verilog HDL

Задание по варианту:

№ варианта	Логический базис	БОЭ
4	NAND	Позиционный шифратор «8 в 3»

Отчет о выполнении заданий части 1:

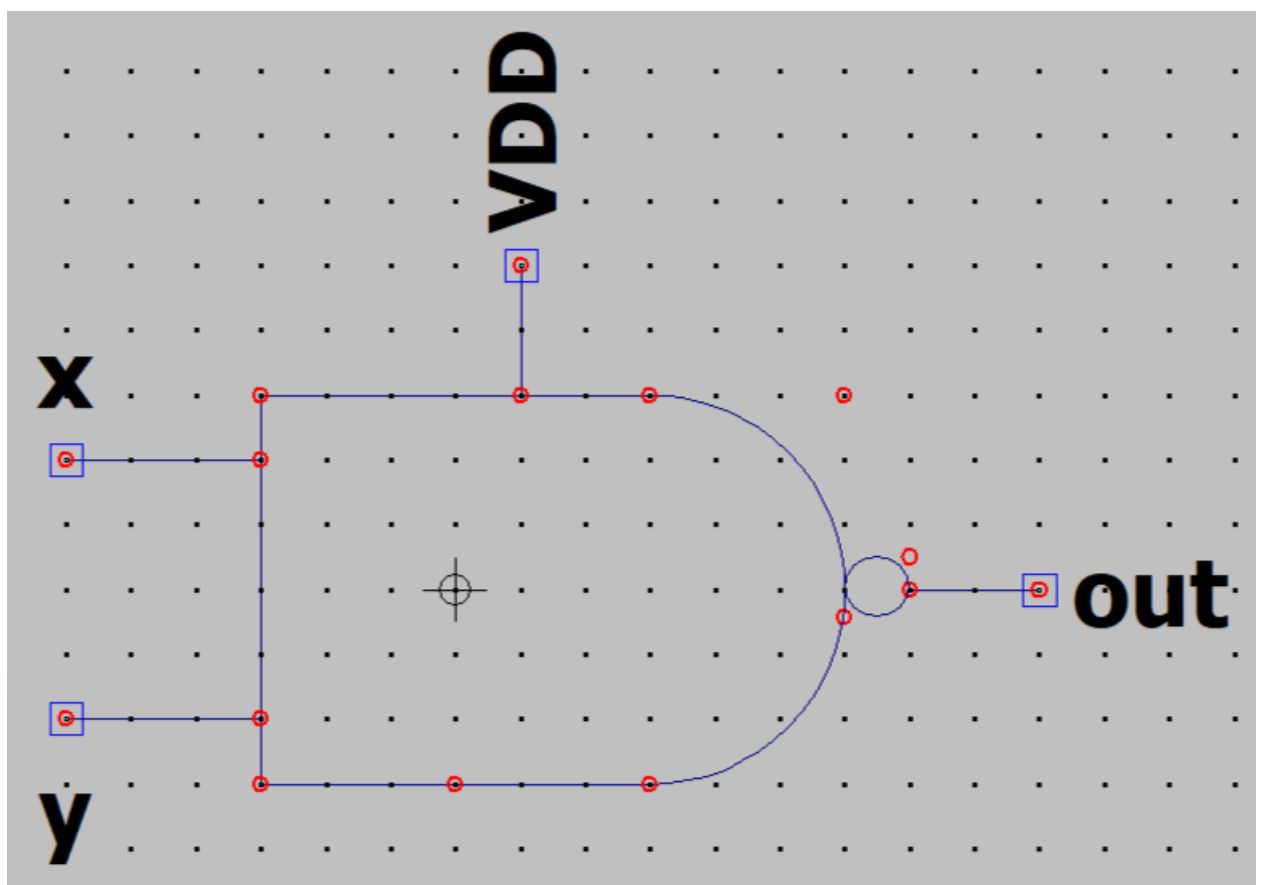
- 1) Схема разработанного вентиля



Используя PMOS и NMOS транзисторы построили NAND. Подвели VDD. Заземлили NMOS M4. Данный компонент соответствует таблице истинности для NAND

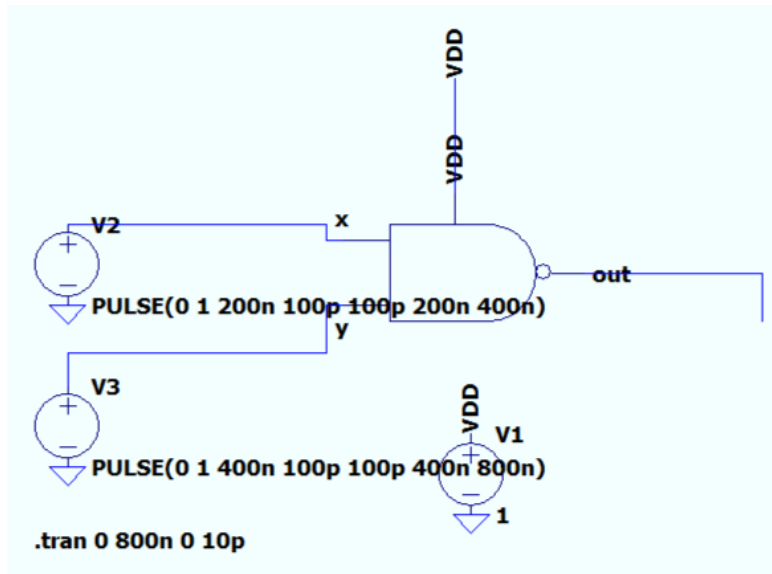
X	Y	OUT
0	0	1
1	0	1
0	1	1
1	1	0

2) Создаем символьный файл.

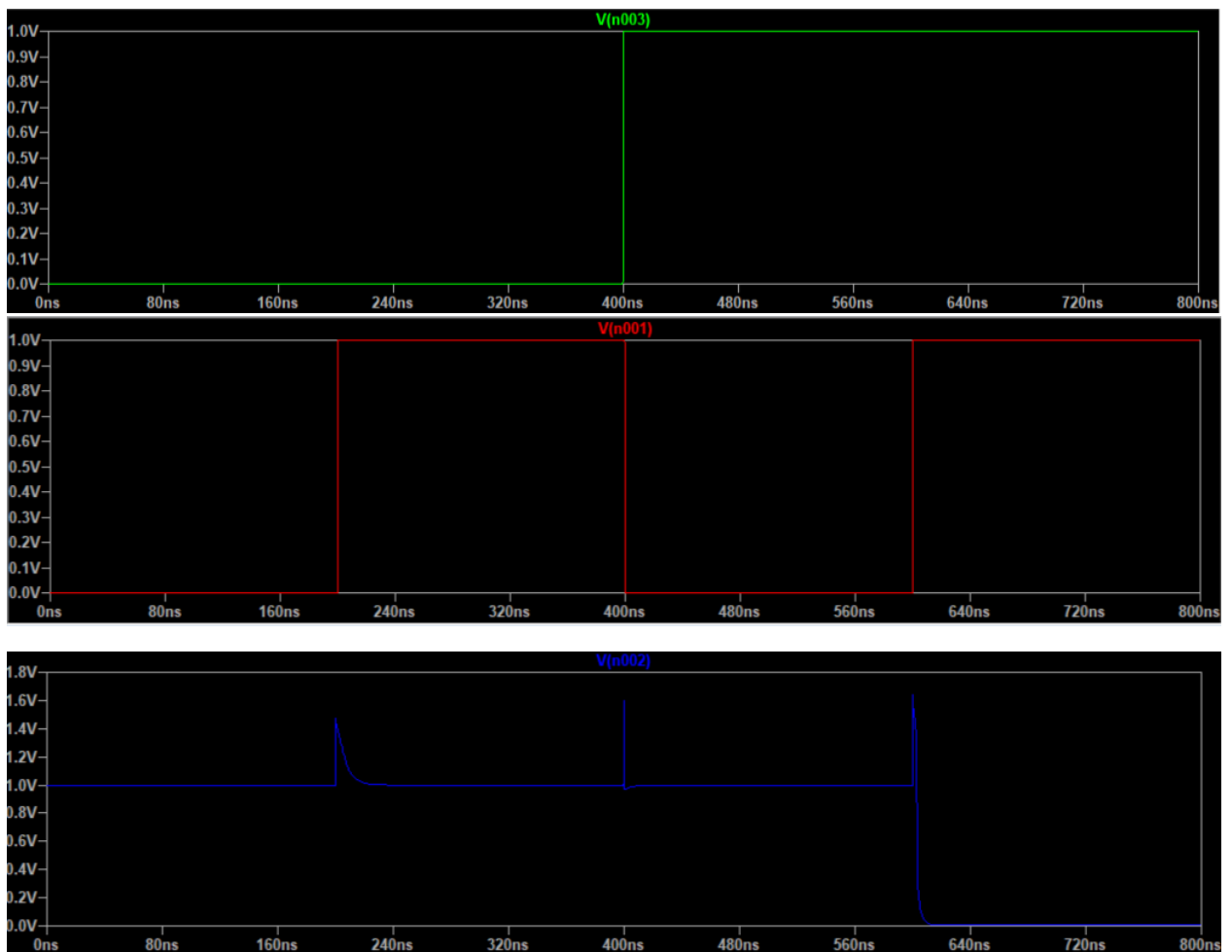


Создали файл .asy с таким же названием как и у схемы разработанного вентиля. Входы и выход называем так же, как было на схеме. Теперь можно использовать новый компонент в проектах. Проверим его работоспособность

3) Построим схему тестирования вентиля.



Построили схему, подключив к компоненту 2 voltage. Один будет половину времени выдавать 0, далее 1. Второй будет поочередно выдавать 0, 1, 0, 1.

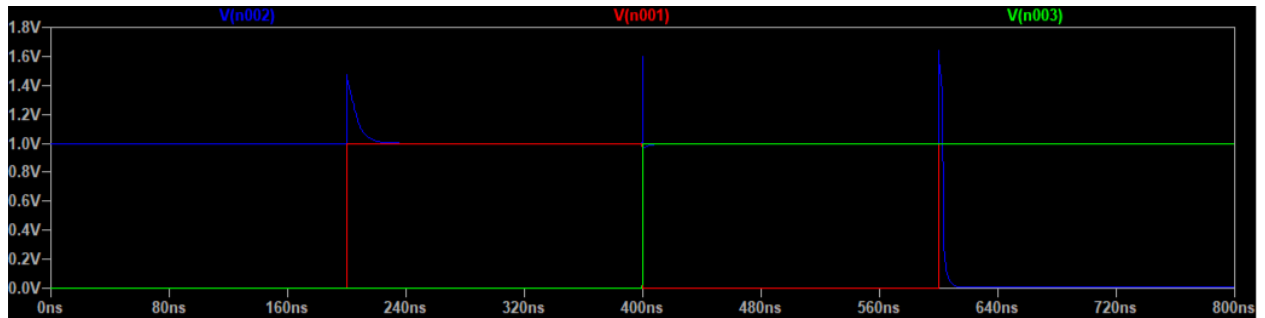


Зеленый – y

Красный – x

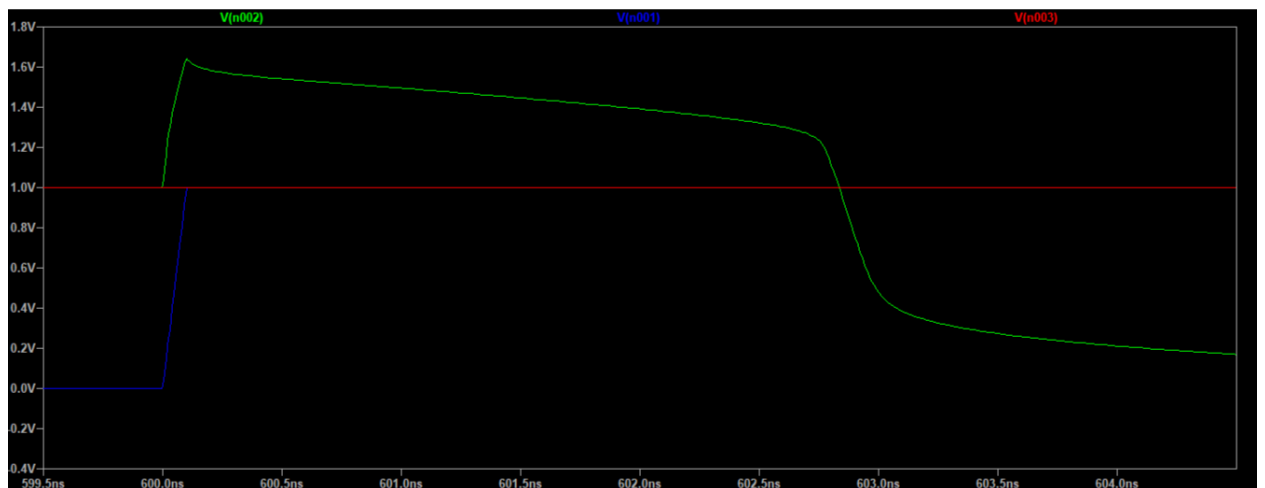
Синий – выходной out

x, y, out на одном графике



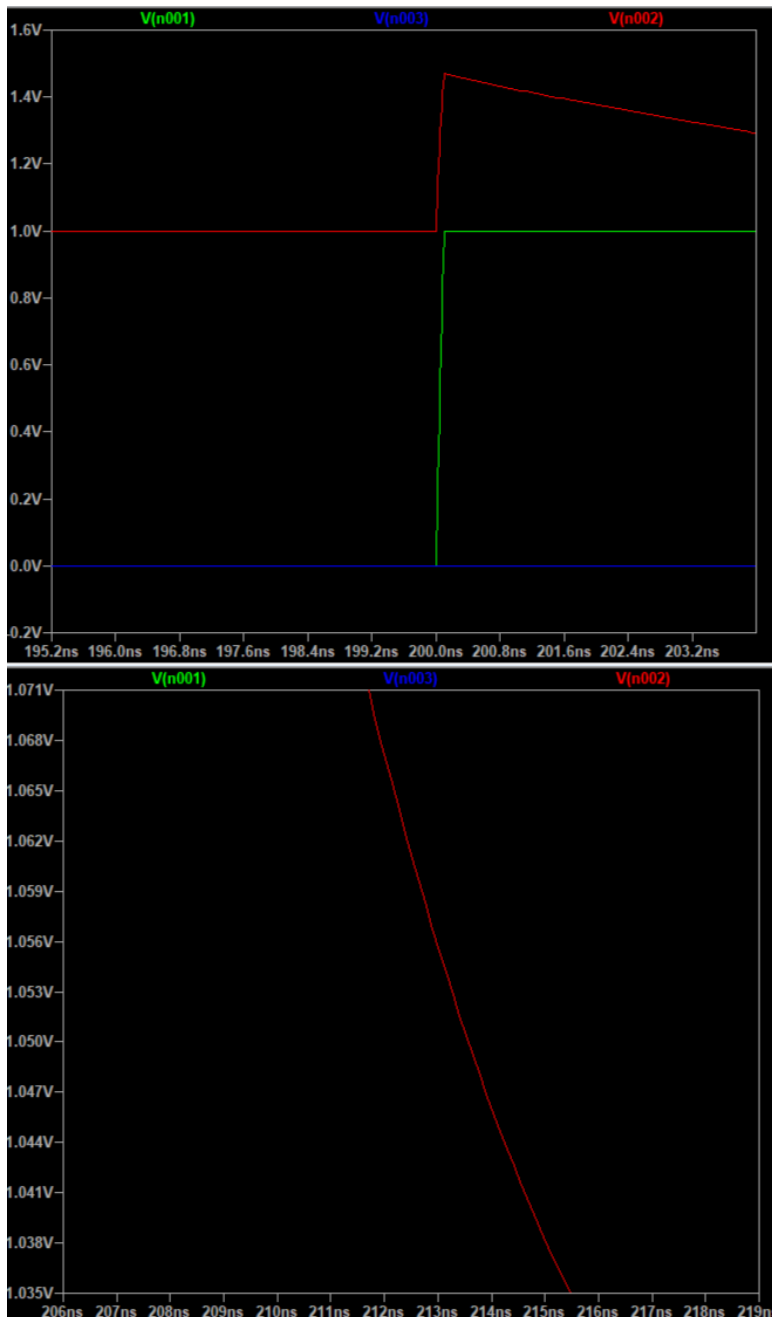
Сравниваем с таблицей истинности и делаем вывод что всё корректно.

- 4) Проведите моделирование работы схемы и определите задержку распространения сигнала через тестируемый вентиль.



Задержка 3ns

5) Определяем максимальную частоту изменения входных сигналов при которых схема сохраняет работоспособность



Начало реакции 200 ns.

Конец восстановления 213 ns.

Время восстановления = 213 ns
– 200 ns = **13 ns**.

Для обычного режима (когда вход меняется каждые полупериоды) нужно, чтобы полупериод $\geq$ Время восстановления.

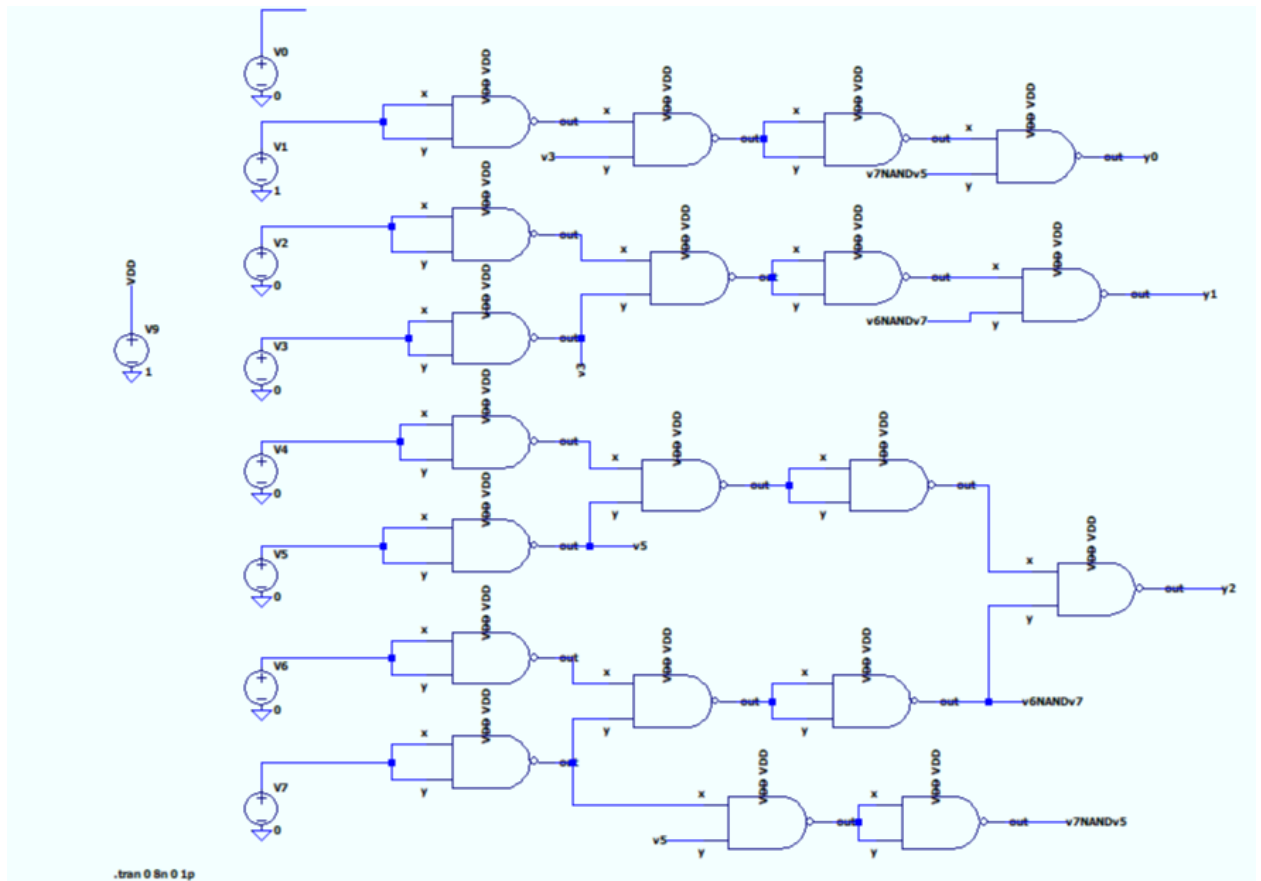
Значит минимальный полный период:

$$T_{\min} = 2 \cdot t = 2 \cdot 13 = 26$$

Максимальная частота:

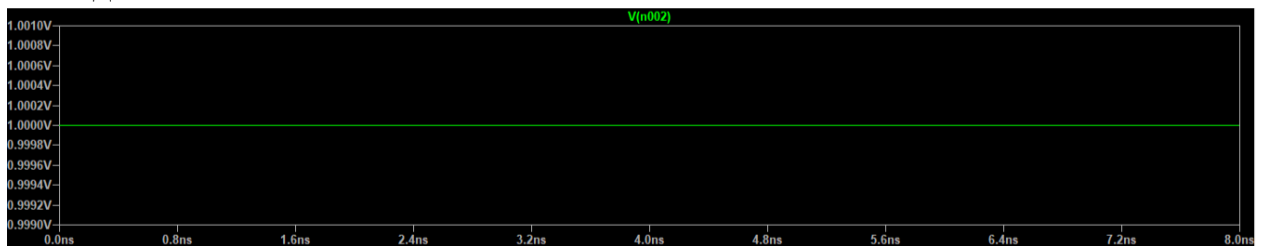
$$f_{\max} = \frac{1}{T_{\min}} = \frac{1}{26} = 38.46 \text{ МГц}$$

6) Постройте БОЭ на базе созданного вентиля согласно варианту задания.

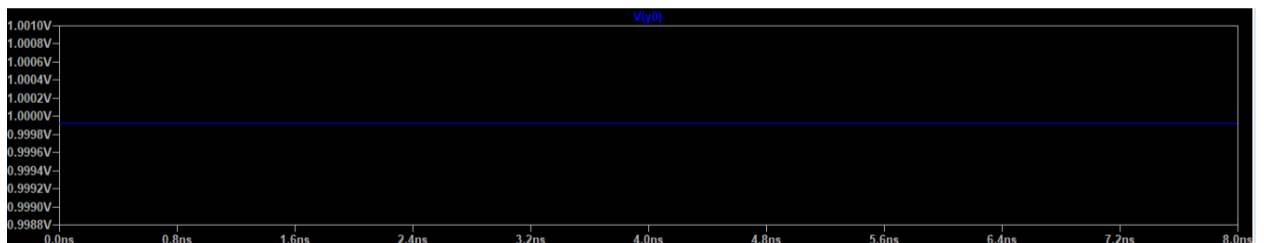


Построим схему БОЭ Шифратор 8 к 3. Протестируем нашу схему. Выставим $V1 = 1$

Выход V1



$$y_0 = 1$$



$$y_1 \text{ и } y_2 = 0$$

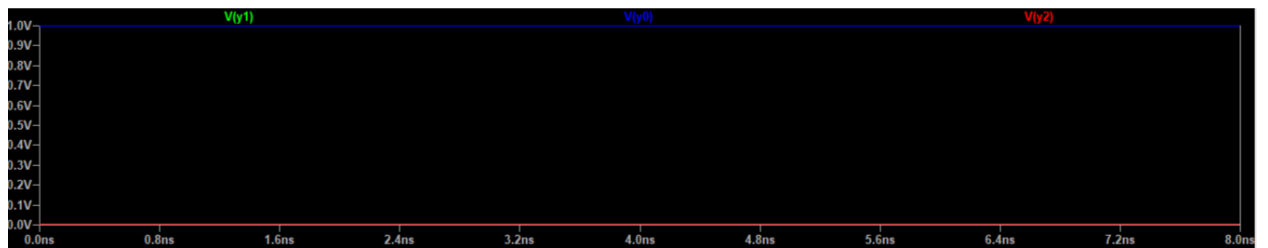
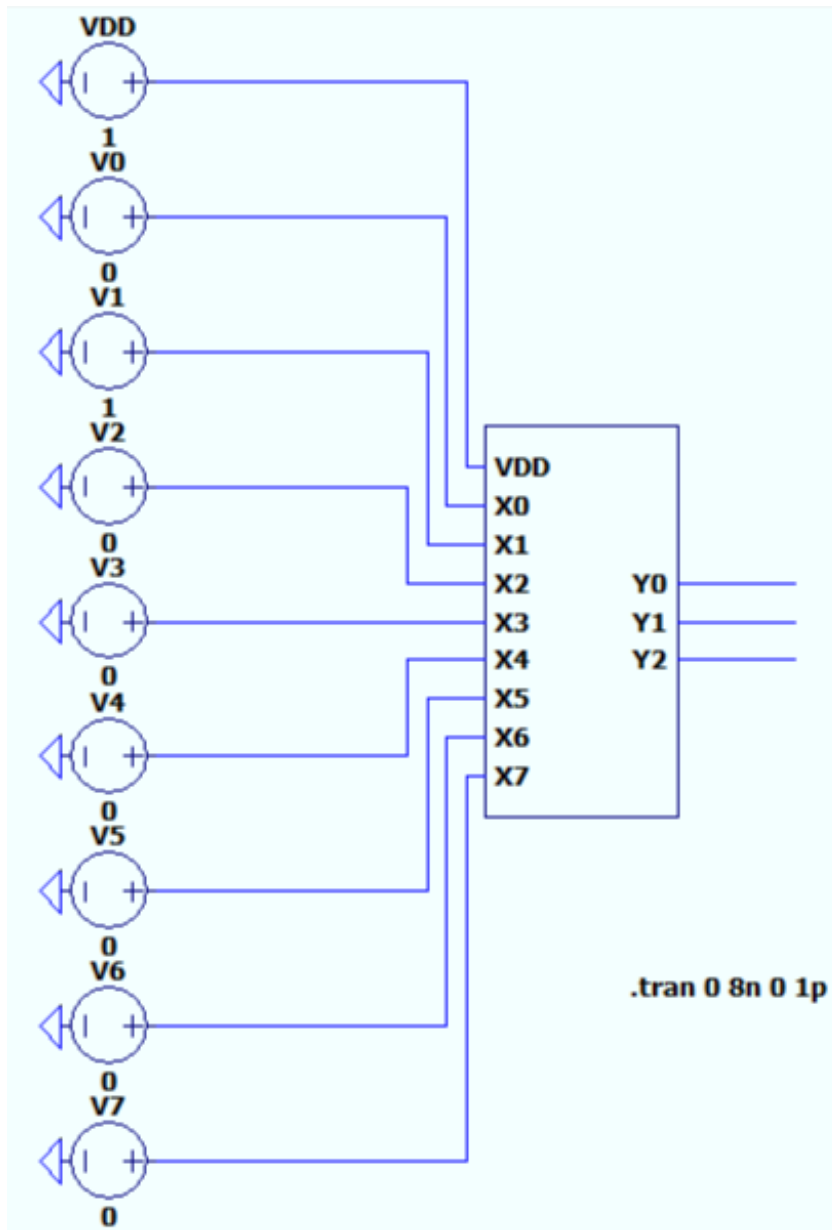


Таблица истинности Позиционного Шифратора 8 к 3

0	1	2	3	4	5	6	7	Y2	Y1	Y0
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

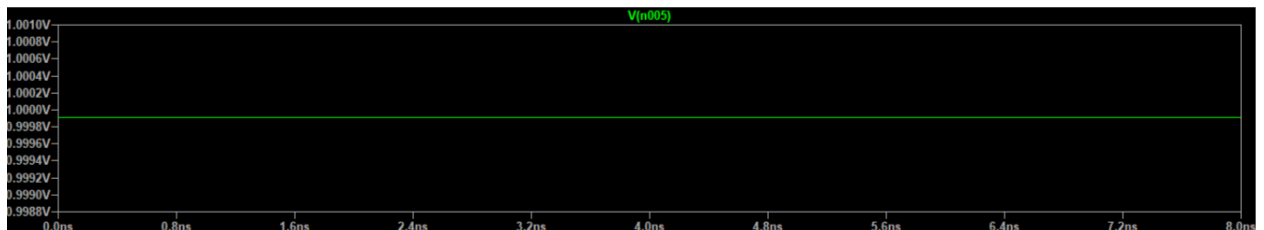
То есть при входных 01000000 мы должны получить 001, что и получилось при тестировании.

7) Создайте символ для построенного БОЭ.

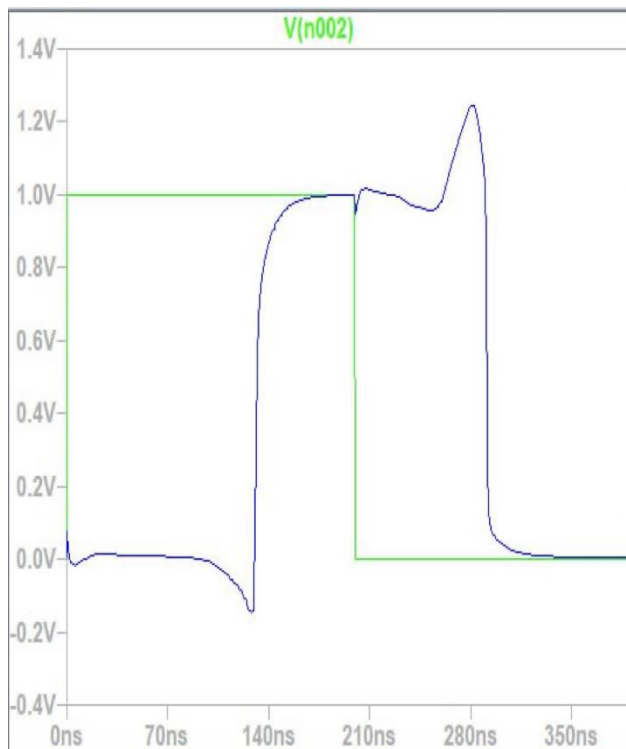


Ставим те же входные данные что и в прошлом шаге

По графику видим, что y_0 стало 1, что и должно было произойти по таблице истинности



- 8) Проведите моделирование работы схемы и определите задержку распространения сигнала через БОЭ.



Задержка БОЭ элемента равна 280ns
- 200 ns = 80 ns

- 9) Определите максимальную частоту изменения входных сигналов, при которой построенная схема сохраняет работоспособность.

$$f_{\max} = \frac{1}{T_{\min}} = \frac{1}{2 \cdot t \cdot 10^{-9}} \approx 6.25 \text{ МГц}$$

Отчет о выполнении заданий части 2:

- 1) Опишите на Verilog HDL на вентильном уровне модуль, реализующий функцию БОЭ(Шифратор 8 к 3) в указанном логическом базисе согласно варианту задания.

...

```
module nand_pos_cypher_8_3(
    input wire [7:0] x,
    output wire [2:0] y
);

    wire nx0, nx1, nx2, nx3, nx4, nx5, nx6, nx7;

    nand g_nx0(nx0, x[0], x[0]);
    nand g_nx1(nx1, x[1], x[1]);
```

```
nand g_nx2(nx2, x[2], x[2]);  
nand g_nx3(nx3, x[3], x[3]);  
nand g_nx4(nx4, x[4], x[4]);  
nand g_nx5(nx5, x[5], x[5]);  
nand g_nx6(nx6, x[6], x[6]);  
nand g_nx7(nx7, x[7], x[7]);
```

```
wire or_x1_x3, or_x5_x7;  
nand g_or_x1_x3_step1(or_x1_x3, nx1, nx3);  
nand g_or_x5_x7_step1(or_x5_x7, nx5, nx7);
```

```
wire not_or_x1_x3, not_or_x5_x7;  
nand g_not_or_x1_x3(not_or_x1_x3, or_x1_x3, or_x1_x3);  
nand g_not_or_x5_x7(not_or_x5_x7, or_x5_x7, or_x5_x7);  
nand g_y0(y[0], not_or_x1_x3, not_or_x5_x7);
```

```
wire or_x2_x3, or_x6_x7;  
nand g_or_x2_x3_step1(or_x2_x3, nx2, nx3);  
nand g_or_x6_x7_step1(or_x6_x7, nx6, nx7);
```

```
wire not_or_x2_x3, not_or_x6_x7;  
nand g_not_or_x2_x3(not_or_x2_x3, or_x2_x3, or_x2_x3);  
nand g_not_or_x6_x7(not_or_x6_x7, or_x6_x7, or_x6_x7);  
nand g_y1(y[1], not_or_x2_x3, not_or_x6_x7);
```

```
wire or_x4_x5, not_or_x4_x5;  
nand g_or_x4_x5_step1(or_x4_x5, nx4, nx5);
```

```

nand g_not_or_x4_x5(not_or_x4_x5, or_x4_x5, or_x4_x5);

nand g_y2(y[2], not_or_x4_x5, not_or_x6_x7);

endmodule

` ` `

```

Эта программа описывает на языке Verilog схему позиционного шифратора 8 к 3, выполненного полностью на элементах NAND. Шифратор 8 к 3 преобразует восемь входных сигналов $x[7:0]$ в трёхразрядный двоичный код $y[2:0]$, который показывает номер активного входа.

Для выхода $y[0]$ формируется логическое выражение, соответствующее дизъюнкции входов $x[1]$, $x[3]$, $x[5]$ и $x[7]$.

Для выхода $y[1]$ — дизъюнкция входов $x[2]$, $x[3]$, $x[6]$, $x[7]$.

Для выхода $y[2]$ — дизъюнкция входов $x[4]$, $x[5]$, $x[6]$, $x[7]$.

2) Разработайте тестовое окружение для созданного модуля.

```

`timescale 1ns / 1ps

module pos_cypher_8_3_ts;

reg [7:0] x_in;

wire [2:0] y_out;

integer i;

reg [2:0] expected_val;

```

```

nand_pos_cypher_8_3 uut (
    .x(x_in),
    .y(y_out)
);

initial begin

    for (i = 0; i < 8; i = i + 1) begin
        x_in = 8'b00000001 << i;
        expected_val = i[2:0];

        #10; // задержка

        if (y_out == expected_val)
            $display("PASS:  input=%b  output=%b  expected=%b",  x_in,  y_out,
expected_val);
        else
            $display("FAIL: input=%b output=%b expected=%b", x_in, y_out, expected_val);
        end

        #10 $stop;
    end

endmodule

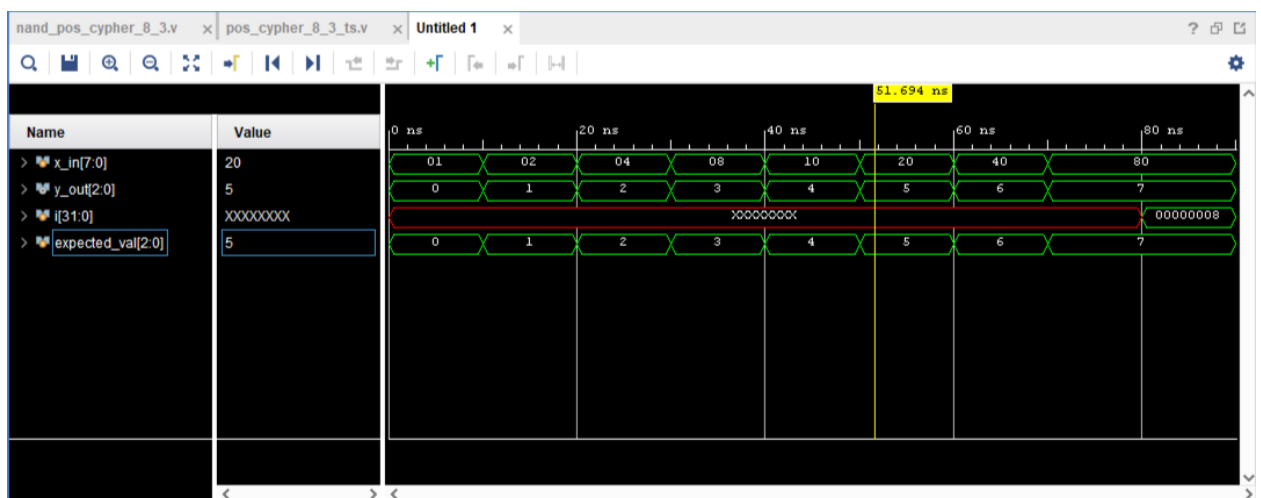
...

```

Файл для тестирования нашей схемы. Проверяем все случаи сдвигая битик единицы влево на один ровно 8 раз. В результате получаем 8 тестов.

3) Проведите моделирование работы схемы.

```
PASS: input=00000001 output=000 expected=000
PASS: input=00000010 output=001 expected=001
PASS: input=00000100 output=010 expected=010
PASS: input=00001000 output=011 expected=011
PASS: input=00010000 output=100 expected=100
PASS: input=00100000 output=101 expected=101
PASS: input=01000000 output=110 expected=110
PASS: input=10000000 output=111 expected=111
$stop called at time : 90 ns : File "C:/Users/AsusAspire 3/Desktop/pos_cypher_8_3_ts.v" Line 38
INFO: [USF-XSim-96] XSim completed. Design snapshot 'pos_cypher_8_3_ts_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:09 ; elapsed = 00:00:11 . Memory (MB): peak = 738.883 ; gain = 3.309
```



Вывод:

В процессе лабораторной работы мы поработали с программой LT SPICE и Vivado. Ознакомились с символами, компонентами и основами работы в LTSpice. Потрогали язык Verilog, это оказалось чуть более проще чем ожидалось. Проекты в Vivado нас не прекращают удивлять. До сих пор не понятно, как правильно удалять файлы из самого приложения. Так как они постоянно сами подтягиваются даже если нажать delete. Но решение было найдено. Просто при любой ошибке, пересоздавать проект. Так мы научились смирению и перепроверке всего 100 раз. Так же посмотрели, как именно работают маленькие схемы и какие бывают неидеальные сигналы.